

Primary sources & further reading

Track 16.2 — TypeScript for AI Applications · 16-ai-tech-stack/tracks/typescript/papers/PAPERS.md · 16-ai-tech-stack/tracks/typescript/resources/RESOURCES.md

Primary Sources — TypeScript for AI Applications

There is no arXiv literature for "how to parse an SSE frame". The primary sources for this track are **specifications and official documentation**, and reading the spec is a genuinely different experience from reading a blog post about the spec.

Read them in this order.

Read these three first

#	Source	Why it matters	Link
1	WHATWG — Server-Sent Events (HTML spec, §9.2)	The actual framing rules: <code>\n</code> vs <code>\r\n</code> vs <code>\r</code> , comment lines, multi-line <code>data:</code> , <code>id:/retry:</code> , and the last-event-id reconnect protocol. Every streaming bug in note 02 is a deviation from this document.	https://html.spec.whatwg.org/multipage/server-sent-events.html
2	WHATWG — Encoding Standard (§TextDecoder, streaming flag)	Why <code>decode(chunk, { stream: true })</code> exists and precisely what the decoder holds back between calls. Ten minutes here permanently fixes the split-UTF-8 bug in your head.	https://encoding.spec.whatwg.org/
3	WHATWG — Streams Standard	<code>ReadableStream</code> , the <code>pull/cancel</code> contract, and the definition of backpressure. The source of truth for every edge runtime's streaming behaviour.	https://streams.spec.whatwg.org/

Type system

Source	Takeaway	Link
TypeScript Handbook — Narrowing	Discriminated unions, type guards, and the <code>never</code> exhaustiveness trick, straight from the source	https://www.typescriptlang.org/docs/handbook/2/narrowing.html
TypeScript Handbook — Template Literal Types	What they can and cannot do; read before you build a type-level parser	https://www.typescriptlang.org/docs/handbook/2/template-literal-types.html
TypeScript 4.9 release notes — <code>satisfies</code>	The original motivation and the exact difference from <code>as</code> and from an annotation	https://www.typescriptlang.org/docs/handbook/release-notes/typescript-4-9.html
TSConfig Reference	Every compiler flag with an example of what it catches. The reference for note 02 §7	https://www.typescriptlang.org/tsconfig/
TypeScript performance wiki	Why your editor got slow, and which type constructs cost the most	https://github.com/microsoft/TypeScript/wiki/Performance

Validation & schemas

Source	Takeaway	Link
Zod documentation	<code>safeParse</code> , <code>z.infer</code> , coercion, refinements, discriminated unions	https://zod.dev/
Zod — JSON Schema conversion	<code>z.toJSONSchema()</code> , the <code>io: "input" \ "output"</code> distinction, and what is unrepresentable	https://zod.dev/json-schema
JSON Schema specification	What the model's tool schema actually means; <code>required</code> , <code>enum</code> , <code>additionalProperties</code>	https://json-schema.org/
<code>zod-to-json-schema</code>	The Zod v3-era converter. Read the README to understand why it is legacy on v4	https://github.com/StefanTerdell/zod-to-json-schema

The SDK layer

Source	Takeaway	Link
Anthropic — Streaming Messages	The full event list (<code>message_start</code> → <code>message_stop</code>), delta types, and the raw SSE payloads	https://platform.claude.com/docs/en/build-with-claude/streaming
Anthropic — Tool use overview	<code>input_schema</code> , <code>tool_choice</code> , <code>tool_result</code> , <code>is_error</code> , parallel tool calls	https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview
Anthropic — Structured outputs	<code>output_config.format</code> and <code>strict: true</code> — schema enforcement at the API level	https://platform.claude.com/docs/en/build-with-claude/structured-outputs
<code>@anthropic-ai/sdk</code> source	Read <code>src/lib/MessageStream.ts</code> for a production-grade version of <code>code/streaming-parser.ts</code>	https://github.com/anthropics/anthropic-sdk-typescript
Claude Agent SDK docs	The agent harness: built-in tools, subagents, hooks, permissions, MCP	https://code.claude.com/docs/en/agent-sdk
Vercel AI SDK docs	<code>generateText</code> / <code>streamText</code> / <code>generateObject</code> , the <code>tool()</code> helper, UI message streams	https://ai-sdk.dev/docs

Runtimes

Source	Takeaway	Link
Cloudflare Workers — Node.js compatibility	Exactly which Node built-ins exist behind <code>nodejs_compat</code> , and which never will	https://developers.cloudflare.com/workers/runtime-apis/nodejs/
Cloudflare Workers — Limits	CPU time, memory, subrequests, and what "wall time is not CPU time" means for streaming	https://developers.cloudflare.com/workers/platform/limits/
Vercel — Functions	Node vs Edge runtime, streaming responses, duration limits by plan	https://vercel.com/docs/functions
Node.js — ECMAScript modules	The ESM/CJS interop rules, <code>import.meta.url</code> , and why extensions are required	https://nodejs.org/api/esm.html

Engineering essays worth the time

Source	Takeaway	Link
Anthropic — Building Effective Agents	Workflow patterns vs. agents, and why most "agents" should be a chain. Language-agnostic but the tool-design advice is the spec for your Zod schemas	https://www.anthropic.com/engineering/building-effective-agents
MDN — Using Server-Sent Events	The readable companion to the WHATWG spec, with <code>EventSource</code> caveats (no custom headers, GET only — which is why everyone uses <code>fetch</code> instead)	https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events

How to read a spec (20 minutes, 3 passes)

1. **Pass 1 (3 min):** find the section that names your bug. Specs have excellent tables of contents and terrible introductions. Skip the introduction.
2. **Pass 2 (12 min):** read the algorithm steps literally, in order. Specs are written as imperative pseudocode. Your parser should be a transcription.
3. **Pass 3 (5 min):** hunt the edge cases — "if the field is absent", "ignore the line". That is where your bug is.

Keep one line of notes per spec section you consulted. The next time the same bug appears, you will find your note before you find the section.

Resources — TypeScript for AI Applications

Learn the type system properly

Resource	Who it is for	Link
TypeScript Handbook	The official tour. Skip to "Narrowing" and "Type Manipulation" if you already write TS daily	https://www.typescriptlang.org/docs/handbook/intro.html
Total TypeScript (Matt Pocock)	Free core-volume tutorials plus the best short-form TS content anywhere. The "Type Transformations" workshop is the fastest route to fluency with generics	https://www.totaltypescript.com/
type-challenges	150+ type-level puzzles, graded. Do the easy and medium tiers; skip "extreme" unless you enjoy suffering	https://github.com/type-challenges/type-challenges
TypeScript Deep Dive (Basarat)	Free book; still the clearest explanation of structural typing and declaration files	https://basarat.gitbook.io/typescript/
ts-reset	A "CSS reset" for TypeScript's less defensible built-in types (<code>JSON.parse</code> returning <code>any</code> , <code>.filter(Boolean)</code>)	https://github.com/mattpocock/ts-reset

The AI SDK layer

Repo / doc	What is inside	Link
Vercel AI SDK	<code>ai</code> + <code>@ai-sdk/*</code> providers. Read <code>examples/</code> in the repo before the docs — they are more current	https://github.com/vercel/ai
AI SDK documentation	<code>streamText</code> , <code>generateObject</code> , tool calling, agent loops, UI hooks	https://ai-sdk.dev/docs
Anthropic TypeScript SDK	Official client. <code>src/lib/MessageStream.ts</code> is worth reading line by line	https://github.com/anthropics/anthropic-sdk-typescript
Claude Agent SDK docs	Agent harness with built-in tools, subagents, hooks, permissions	https://code.claude.com/docs/en/agent-sdk
MCP TypeScript SDK	Server and client for Model Context Protocol; takes Zod schemas for tool definitions directly	https://github.com/modelcontextprotocol/typescript-sdk
ai-chatbot (Vercel)	A complete, production-shaped Next.js chat app: streaming, tools, auth, persistence	https://github.com/vercel/ai-chatbot

Validation & error handling

Resource	Why	Link
Zod	The default. Read the docs on <code>safeParse</code> , <code>discriminatedUnion</code> , and JSON Schema	https://zod.dev/
Valibot	Zod's API with a much smaller bundle, via modular imports. Worth it when bundle size is a real constraint (edge, browser)	https://valibot.dev/
ArkType	Parses TypeScript-syntax type strings at runtime; the fastest of the three. Younger ecosystem	https://arktype.io/
Standard Schema	The shared interface Zod/Valibot/ArkType all implement, so libraries can accept any of them	https://github.com/standard-schema/standard-schema
neverthrow	Minimal <code>Result<T, E></code> with combinators, if the 1-line type alias stops being enough	https://github.com/supermacro/neverthrow
Effect	A full effect system: typed errors, dependency injection, structured concurrency, retries. Powerful and a large commitment — adopt deliberately, not incidentally	https://effect.website/

Runtimes & deployment

Resource	Why	Link
Cloudflare Workers docs	The reference edge runtime. Read the Node.js-compat and Limits pages before you port anything	https://developers.cloudflare.com/workers/
Vercel Functions docs	Node vs Edge runtime, streaming, duration limits	https://vercel.com/docs/functions
Bun docs	Runtime, bundler, test runner, package manager in one binary	https://bun.sh/docs
Deno docs	Permissions model, web-standard APIs, npm compatibility	https://docs.deno.com/
Hono	Tiny web framework that runs unmodified on Node, Bun, Deno, Workers and Vercel Edge. The pragmatic choice for an AI backend that must be portable	https://hono.dev/
Nitro / h3	The other portable-server answer; powers Nuxt	https://nitro.build/

Toolchain

Tool	Link
pnpm — strict, fast, content-addressed package manager	https://pnpm.io/
Vitest — the test runner to use with TypeScript in 2026	https://vitest.dev/
Biome — eslint + prettier replacement, one Rust binary	https://biomejs.dev/
typescript-eslint — if you need rules Biome does not have	https://typescript-eslint.io/
tsx — run a <code>.ts</code> file directly. Remember: it does not typecheck	https://github.com/privatenumber/tsx
tsup — bundle a TS library without a webpack config	https://tsup.egoist.dev/
Changesets — versioning and changelogs for monorepos	https://github.com/changesets/changesets

Observability for AI apps

Tool	Why	Link
OpenTelemetry JS	Vendor-neutral tracing. The AI SDK emits OTel spans for model calls	https://opentelemetry.io/docs/languages/js/
Langfuse	Open-source LLM tracing with a solid TS SDK; self-hostable	https://langfuse.com/
LangSmith	See the sibling track <code>../langsmith/ [16-ai-tech-stack/tracks/langsmith]</code>	https://docs.smith.langchain.com/

Communities & keeping current

- TypeScript release notes — the only reliable way to hear about new flags: <https://devblogs.microsoft.com/typescript/>
- Matt Pocock's newsletter/videos — short, high signal: <https://www.totaltypescript.com/>
- r/typescript — <https://www.reddit.com/r/typescript/>
- Anthropic engineering blog — <https://www.anthropic.com/engineering>

- AI SDK changelog — the API moved a lot in v5; read it before upgrading:
<https://github.com/vercel/ai/releases>

Sibling tracks

- `../python/` [16-ai-tech-stack/tracks/python] — the other half of the wire
- `../claude-code/` [16-ai-tech-stack/tracks/claude-code] — the Claude Agent SDK in anger
- `../langchain/` [16-ai-tech-stack/tracks/langchain] and `../langgraph/` [16-ai-tech-stack/tracks/langgraph] — both have TypeScript ports; note 02 §5 is the context for deciding whether you want one